SBER TECH | PLATFORM V

Временные таблицы для PostgreSQL

Почему это важно для платформы 1С и что можно улучшить?

Павел Селезнёв

PG BootCamp Russia 2026 Moscow
PGBootCamp.ru

Павел Селезнев

В команде СУБД Pangolin с 2021 года

Опыт разработки с использованием MySQL, Firebird, Oracle, MSSQL

С 2011 по 2021 год участвовал в разработке браузеров Рамблер и Netbox

Почему платформа 1С?

Поговорим о временных таблицах

Сценарии использования и особенности работы

Почему используются временные таблицы в конфигурациях на платформе 1С

Статистика использования

Структура патча от компании 1С

Патч для временных таблиц от компании 1С с доработками команды Pangolin

Сценарии использования временных таблиц

Основные

- Аналитика на основе долгого запроса
- Оптимизация – разбиваем сложный запрос на этапы

Дополнительные

- Помощь в миграции структуры таблицы
- Формирование данных для отчетов
- Платформа 1С – улучшение читаемости запросов
- Создание хранилища переменных

MS SQL

Глобальные временные таблицы – создаем структуру один раз для всех пользователей

```
create table ##tablename
```

Локальные временные таблицы

```
Create table #tablename
```

Tempdb – база данных для временных файлов MS SQL, не реплицируется, состоит из файлов, есть право на коннект, создается заново при перезапуске

PostgreSQL

Поддерживает только локальные временные таблицы

Параметр `temp_tablespaces` определяет физическое пространство, где размещать временные таблицы

Oracle

Глобальные временные таблицы

`create global temporary table`

Локальные временные таблицы появились в oracle18c

`create private temporary table`

Некоторые ограничения:

- Нельзя использовать вторичные ключи
- Временная таблица не может быть партицирована, кластеризована
- Нельзя использовать подсказку PARALLEL и параллельные запросы не поддерживаются во временных таблицах
- Не поддерживаются распределенные транзакции

PostgreSQL

Данные временных таблиц доступны только для одного процесса

Временные таблицы могут быть партицированы

СУБД Tantor / Postgres Pro

- Параллельный SELECT
- Параллельный INSERT

mySQL

https://dev.mysql.com/doc/refman/9.6/en/internal-temporary-tables.html

Отличная документация, описывает:

- Неявные временные таблицы
- Как работает механизм хранения временных таблиц и как управлять распределением при выгрузке данных из памяти на диск при превышении
- Физическую структуру хранения таблиц

Postgres

Параметры `temp_buffers` определяет объем памяти, используемый как для временных таблиц, так и для неявных объектов

Путь к временным таблицам `<pgdata>/base/pgsql_tmp`

PostgreSQL. Что получаем?

Проблемы

- Нужно создавать временную таблицу в рамках сессии
- Замусоривание системного каталога при частом создании временных таблиц

Что есть сейчас из решений?

`pgtt-rsl` – «глобальные» таблицы для решения проблемы замусоривания системного каталога

`pgtt` – «глобальные» таблицы для решения проблемы разработчика

Расширение PGTT-RSL

Реализация на основе unlogged-таблицы и через включение безопасности на уровне строк

Плюсы	Минусы
Таблица в любой схеме	Замедление из-за использования ROW security
Не замусориваем системный каталог	Background worker чистит строки
	Есть вопросы безопасности

Расширение PGTT

Первый запрос с участием имени таблицы создаст временную таблицу в схеме `pg_temp` из шаблона в схеме `pgtt_schema`

```
SELECT * FROM pgtt_schema.test_tt
```

=> Реально перенаправляется на схему pg_temp

Плюсы	Минусы
Не нужно в коде думать про create temp table	Также замусориваем системный каталог
Позволяет использовать все возможности временных таблиц	

Особенности временных таблиц

- Безопасность
- Временные таблицы не пересекаются по блокировкам
- Можно переиспользовать данные временных таблиц
- Не нагружаются механизмы репликации (WAL-файлы, канал до реплики, CPU реплики)

Альтернативы для разработчика

	Минус	Плюс
Расширение pgvariables	Нет аудита, слабая безопасность	Не требуется работать с таблицами
Unlogged table	Данные видны всем, нужно вводить поле session	Не требуется пересоздавать таблицу
Представления Views	Не дает помощь планировщику запросов	Визуально упрощает запрос
Материализованное представление	Необходимо периодически делать синхронизацию данных	Визуально упрощает запрос
Общие табличные выражения CTE	Необходимо ручное переписывание кода	Ускоряет работу запроса

Платформа 1С

Почему используются временные таблицы

Почему сложны SQL-запросы от платформы 1С

Объекты прикладного разработчика	Платформа 1С SDBL	Объекты базы данных
Справочник Регистр накопления Счет	Метаданные 1С – слой изоляции между приложением и особенностями базы данных	**Регистр накопления:** • таблица движений • таблица остатков • таблица оборотов итд.

Простейшая модель

Регистр накопления «Движение товаров»

- Ссылка на товар (имя товара)
- Количество со знаком плюс или минус
- Дата операции

Отчет по движению с фильтрацией по товару

```
ВЫБРАТЬ
    ОстаткиТоваровОстаткиИОбороты.Товар КАК Товар,
    ОстаткиТоваровОстаткиИОбороты.КоличествоНачальныйОстаток КАК КоличествоНачальныйОстаток,
    ОстаткиТоваровОстаткиИОбороты.КоличествоОборот КАК КоличествоОборот,
    ОстаткиТоваровОстаткиИОбороты.КоличествоПриход КАК КоличествоПриход,
    ОстаткиТоваровОстаткиИОбороты.КоличествоКонечныйОстаток КАК КоличествоКонечныйОстаток,
ИЗ
    РегистрНакопления.ОстаткиТоваров.ОстаткиИОбороты КАК ОстаткиТоваровОстаткиИОбороты
ГДЕ
    Товар = &Товар
```

Запрос к отчету по движению

```
SELECT TL. F1d61RRef,
        TA. Fld62Initia1Balance_,
        TL. F1d62Receipt_,
        T1. F1d62Expense_
        T1. F1d62F1na18alance_,
        T1.F1d61RRef,
        TZ. Description, T7._Descríption
FROM (
        SELECT T2. F1d61RRef AS F1d61RRef,
                CAST AS AS NUMERIC(22, O)) AS F1d62Receipt_,
                CAST (SUM(T2. F1d62Expense._) AS NUMERIC(22, 0)) AS FId62Expense_,
                CAST (SUM(T2. F1d62Balance_) AS NUMERIC (34, 0)) AS Flde2InitialBalance_
                CAST (SUMT2. F1d62Balance. + T2. F1d62Receipt. - T2. F1d62Expense.) AS NUMERIC(35, 0)) AS F1d62FinalBalance.
        FROM (
                SELECT
                        CAST (CAST (SUM (0.0) AS NUMERIC (1, O)) AS NUMERIC (28, 0)) AS FId628alance_,
                        CAST (SUM (CASE WHEN T3. Recordkind = 0.0 THEN T3._FId62 ELSE 0.0 END) AS NUMERIC (16, 0)) AS Fid62Recept_.
                        FAST (SUM(CASE WHEN T3. Recordkind = 0.0 THEN 0.0 ELSE T3._Fld62 END) AS NUMERIC (16, 0)) AS F1d62Expense_
                FROM dbo._AccumRo60 T3
                WHERE T3. Period > @P1 AND T3. Period <= QP2 AND T3. Active = 0x01
                GROUP BY T3._F1d61RRef
                HAVING (CAST(CAST (SUM (0.0) AS NUMERIC (1, O)) AS NUMERIC (28, 0))) 0.0 OR (CAST (SUM(CASE WHEN T3. Recordkind = 0.0 THEN T3._FId62 ELSE 0.0 END) AS
                        NUMERIC (16, O))) 0.0 OR (CAST (SUM(CASE WHEN T3. Recordkind = 0.0 THEN 0.0 ELSE T3._F1d62 END) AS NUMERIC (16, O))) - 0.0
UNION ALL
SELECT T4. F1d61RRef AS FIdGiRRef,
        CAST (SUM(T4, Fld628alance_) AS NUMERIC (28, 0)) AS Fid628alance_,
        CAST(CAST (SUM(T4. F1d62Receipt._) AS NUMERIC (7, 0)) AS NUMERIC (16, 0)) AS F1d62Receipt_, CAST(CAST (SUMT4. F1d62Expense_) AS NUMERIC (7, 0)) AS NUMERIC (16, 0)) AS
FROM (
        SELECT T5._ FIdeRRef AS FIdGIRRef
                CAST (SUM(T5._ F1d62) AS NUMERIC (22, 0)) AS F1d628alance_,
                OP3 AS F1d62Receipt_,
                OP4 AS F1d62Expense_
        FROM dbo._AccumRgT63 T5
        WHERE T5._Period = @P5 AND (T5.Fld62 <> @P6)  AND (T5._Fld62 <> 0)
        GROUP BY T5._Fld61RRef
        ....
```

Оптимизируем для большого количества данных

Передаем список товаров в качестве параметра в виртуальную таблицу

```
ВЫБРАТЬ
   ОстаткиТоваровОстаткиИОбороты.Товар КАК Товар,
   ОстаткиТоваровОстаткиИОбороты.КоличествоНачальныйОстаток КАК КоличествоНачальныйОстаток,
   ОстаткиТоваровОстаткиИОбороты.КоличествоОборот КАК КоличествоОборот,
   ОстаткиТоваровОстаткиИОбороты.КоличествоПриход КАК КоличествоПриход,
   ОстаткиТоваровОстаткиИОбороты.КоличествоКонечныйОстаток КАК КоличествоКонечныйОстаток,
ИЗ
   РегистрНакопления.ОстаткиТоваров.ОстаткиИОбороты(,,,, Товар В (&СписокТоваров)) КАК
ОстаткиТоваровОстаткиИОбороты
```

Менеджер временных таблиц

Приближенный к реальности сценарий

```
ВЫБРАТЬ
	Номенклатура.Ссылка КАК Товар
ПОМЕСТИТЬ ВТ_Товары
ИЗ
	Справочник.Номенклатура КАК Номенклатура
ГДЕ
	Номенклатура.Ссылка В(&МассивТоваров);
```

```
ВЫБРАТЬ
  ОстаткиТоваровОстаткиИОбороты.Товар КАК Товар,
  ОстаткиТоваровОстаткиИОбороты.КоличествоНачальныйОстаток КАК КоличествоНачальныйОстаток,
  ОстаткиТоваровОстаткиИОбороты.КоличествоОборот КАК КоличествоОборот,
  ОстаткиТоваровОстаткиИОбороты.КоличествоПриход КАК КоличествоПриход,
  ОстаткиТоваровОстаткиИОбороты.КоличествоКонечныйОстаток КАК КоличествоКонечныйОстаток,
ИЗ
  РегистрНакопления.ОстаткиТоваров.ОстаткиИОбороты(,,,, Номенклатура В (ВЫБРАТЬ Товар ИЗ ВТ_Товары) ) КАК
ОстаткиТоваровОстаткиИОбороты
```

Статистика использования временных таблиц

Как

Тестовый сценарий

Используется для проверки гипотез по улучшению производительности 1С внутри команды СУБД Pangolin

- Создан на базе ERP – самой массовой конфигурации
- 1 час – время выполнения сценария
- Избыточное количество сессий – 1000 (стандартный сценарий – 50 пользователей)
- Выполняется расширением ТестЦентр для имитации виртуальных пользователей

3 314 509

select-запросов

466 846

операций по созданию временных таблиц

С чем можно сравнить?

1С-сценарий	Тестовые сценарии команды Pangolin	Аудитория
~15%	0.0009%	?

Вставки во временную таблицу

1 657 878

операций по вставке данных
`insert into .. from select.`

из них

231 889

с подзапросом из временной таблицы

Индексы и анализ статистики

358 542

операций по созданию индекса для временной таблицы

1 223 186

операций ANALYZE для временных таблиц

Быстрая очистка

Для быстрой очистки вместо TRUNCATE <имя таблицы>
используется расширение fasttrun
с функцией **SELECT FASTTRUNCATE (<имя таблицы>);**

1 051 354

вызовов функции быстрой очистки

DDL

466 846
CREATE TEMPORARY TABLE

358 542
CREATE INDEX

1 223 186
ANALYZE

1 051 354
FASTTRUNCATE

...

SELECT/DML

3 314 509
SELECT запросов

1 657 878
INSERT запросов

Что можно сделать в PostgreSQL

Патч 1С + доработки Pangolin

Состав патча от 1С

Изменения работы оптимизатора

Улучшения про временные таблицы

Расширения (~ 50% патча):

Совместимость:

- fulleq добавляет оператор ==, где, если оба операнда NULL, то тоже истина
- mchar – добавляет тип MCHAR, аналог MSSQL, использует ICU

Обслуживание:

- fasttrun – добавляет функцию быстрой очистки
- online_analyze – проводит анализ данных после INSERT/UPDATE/DELETE

Оптимизатор:

- plantuner – расширение для отключения индексов при построении плана запросов

Отмена сообщений об Invalidate SendSharedInvalidMessages

476 227 операций drop table

Необходимо уведомлять другие бэкенды об изменениях системного каталога, в том числе и на операции drop table

Отмена блокировок на временную таблицу при построении запроса

src/backend/storage/lmgr/lock.c

```
bool
LockHeldByMe(const LOCKTAG *locktag,
LOCKMODE lockmode, bool orstronger)
{
    …
     if ((locallock && locallock-
>nLocks > 0) ||
     IsLockCanBeSkipped(locktag,false
)
     )
        return true;
```

55 %

select-запросов содержат обращения к временным таблицам

(1 840 778 из 3 314 509)

Изменения системного каталога

В среднем 20 полей во временной таблице

466 609 таблиц займут **1.6 гб** в каталоге

Распределение полей во временных таблицах

	до 7 полей	до 13 полей	до 26 полей	до 50 полей	до 160 полей
Процент таблиц	25%	50%	75%	93%	99%

Информацию о структуре временной таблицы храним только в памяти backend

Решаемые проблемы

- Раздувается системный каталог => autovacuum вынужден обрабатывать системный каталог, но он не блокирует чтение и запись
- Нагрузка на WAL
- Придется делать vacuum full
- Работа с системным каталогом тоже занимает время

Новые проблемы, решенные Pangolin

- Могут быть пересечения по OID, доработали **AddVirtualOid/CheckVirtualOid**
- Не видны временные таблицы, доработали heap_beginscan/heap_endscan, index_beginscan_bitmap для поддержки запросов вида **SELECT relname FROM pg_class WHERE relpersistence = 't'**
- Удаление функции для функциональных индексов может привести к ошибке => доработали **deleteObjectsInList =>check_dependency => есть ли oid функции в индексе временной таблицы**

Количество строк

В среднем во временную таблицу вставляем

1.34 страниц **(118** строк**)**

Распределение полей во временных таблицах

Во временной таблице	0 строк	до 30 строк	до 2500 строк	MAX строк
	33%	61%	4%	277 560

Отложенная запись временных таблиц

В оригинальном PostgreSQL, даже если таблица помещается в память, мы все равно делаем файловые операции (создание, проверка наличия и т.д.)

Решение:
работать с файловой системой, только когда не влезаем в temp_buffers

```
Код, который вызвал проблемы:
bool
mdexists(SMgrRelation reln, ForkNumber forknum)
{
    …

  return (mdopenfork(reln, forknum, EXTENSION_RETURN_NULL) != NULL);
```

Шаблон имени для временной таблицы `<OID_таблицы>_<backend_ID>`

Что у нас есть

- Быстрая работа конфигураций на платформе 1С невозможна без использования временных таблиц
- 1С использует нехарактерным образом временные таблицы – на 3 млн SELECT-запросов приходится 466 846 операций по созданию временных таблиц

Решение: не делиться информацией

- Хранить метаданные локальной временной таблицы только в памяти бэкенда
- Минимизировать файловые операции при работе с данными временной таблицы

Еще кое-что

1 231 байт

средний размер текста запроса 1С

```
    SELECT DISTINCT t1._q_001_f_000, t1._q_001_f_001rref, t1._q_001_f_002_type, t1._q_001_f_002_rtref, t1._q_001_f_002_rrref
    FROM
    pg_temp.tt238 t1
UNION
    SELECT DISTINCT t2._q_001_f_000, t2._q_001_f_001rref, t3._fld15405_type, t3._fld15405_rtref, t3._fld15405_rrref
    FROM pg_temp.tt238 t2
    INNER JOIN _document1114 t3 ON (
    t2._q_001_f_002_type = '\\010'::bytea
    AND t2._q_001_f_002_rtref = '\\000\\000\\004z'::bytea
    AND t2._q_001_f_002_rrref = t3._idrref
    )
    WHERE (t3._fld2830 = CAST(0 AS NUMERIC))
UNION
    SELECT DISTINCT t4._q_001_f_000, t4._q_001_f_001rref, '\\010'::bytea, '\\000\\000\\004z'::bytea, t6._idrref
    FROM pg_temp.tt238 t4
    INNER JOIN _document1114 t5 ON (
    t4._q_001_f_002_type = '\\010'::bytea
    AND t4._q_001_f_002_rtref = '\\000\\000\\004z'::bytea
    AND t4._q_001_f_002_rrref = t5._idrref
    )
    INNER JOIN _document1114 t6 ON (
    t5._fld15405_type = t6._fld15405_type
    AND t5._fld15405_rtref = t6._fld15405_rtref
    AND t5._fld15405_rrref = t6._fld15405_rrref
    )
    AND (t6._fld15432rref = '\\242\\217x/\\261\\261\\3703@\\245\\025\\241\\025\\304l,'::bytea )
    WHERE ((t5._fld2830 = CAST(0 AS NUMERIC)))
    AND (t6._fld2830 = CAST(0 AS NUMERIC))
UNION
    SELECT DISTINCT DATE_TRUNC('month', t7._period), t7._fld57027rref, t7._fld57028_type, t7._fld57028_rtref, t7._fld57028_rrref
    FROM _accumrg57026 t7
    WHERE ((t7._fld2830 = CAST(0 AS NUMERIC)))
    AND (
    ( t7._recordertref = '\\000\\000\\005\\177'::bytea
    AND t7._recorderrref IN ('\\237\\215\\000pv\\037\\315\\367\\021\\360\\332cq\\377|\\352'::bytea)
    )
    AND t7._fld57043 = TRUE
    )
```

1 537 байт

Быстрые блокировки

Нативная интеграция

6

медиана количества объединений таблиц в запросах

30

«максимум» таблиц в запросах

доклад

Михаила Гелемеева

https://ul25.nastachku.ru/1c-на-postgresql-что-делать-с-lwlock

Литература

Обсуждение в сообществе:

- Fast Temporary tables https://www.postgresql.org/message-id/flat/20160301182500.2c81c3dc%40fujitsu
- Global Temporary tables https://www.postgresql.org/message-id/flat/73954ab7-44d3-b37b-81a3-69bdcbb446f7%40postgrespro.ru
- Global Temporary tables https://www.postgresql.org/message-id/flat/1A1A6EDC-D0EC-47B0-BD21-C2ACBAEA65E4%40alibaba-inc.com
- Declarative variable syntax https://commitfest.postgresql.org/patch/1608/

Другое:

- Регистр накопления https://infostart.ru/1c/articles/1061227/, https://its.1c.ru/db/metod8dev/content/2726/hdoc
- Правила использования временных таблиц https://its.1c.ru/db/v8std/content/777/hdoc

SBER TECH

PG BootCamp Russia 2026 Moscow
PGBootCamp.ru

Сообщество Pangolin

Спасибо за внимание!

Platform V Pangolin DB

Seleznev.P.N@sbertech.ru